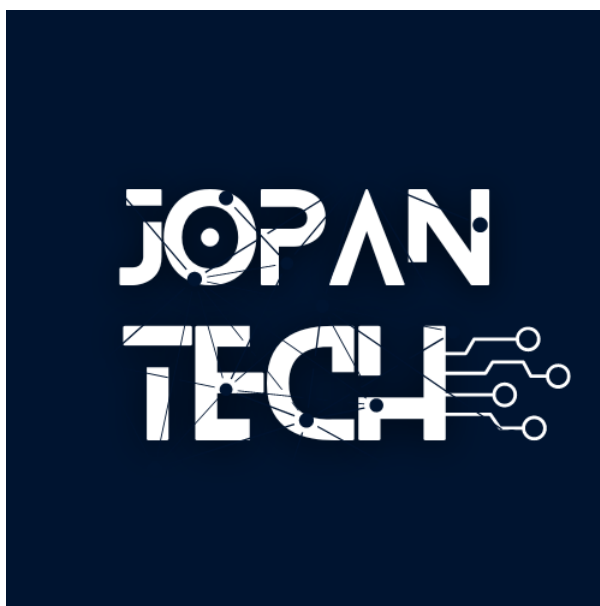




Token-0x Audit Report

Prepared by: jopantech

Table of Contents

- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
 - [Roles](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
- [High](#)
- [Medium](#)

Protocol Summary

Token-0x, A secure and cheap base ERC20 implementation, which follows the ERC20 standard. Token-0x has implemented all the necessary functions required to be a compliant ERC20 token but in different way. Token-0x achieves the secure and cheap operations by using a combination of Solidity and Yul in the base implementation. User can use it as ERC20 token like openzeppelin implementation.

Disclaimer

The jopantech team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

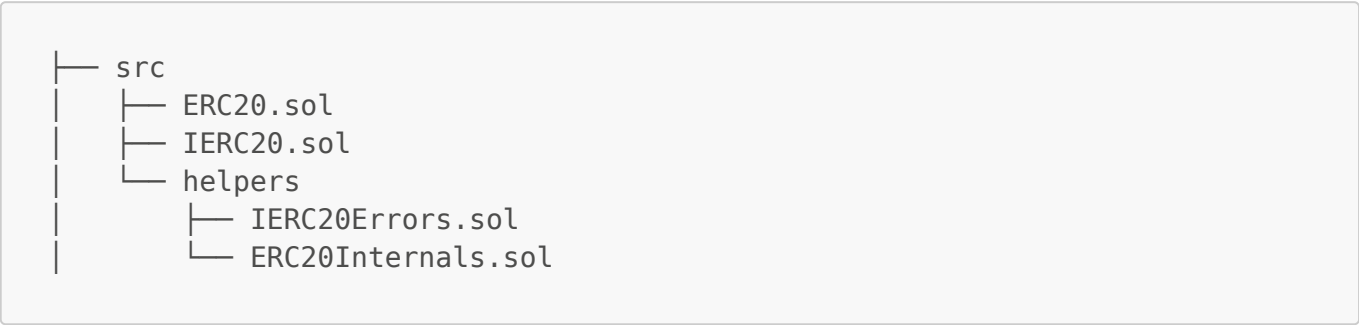
Risk Classification

Impact				
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope



Roles

There is only 1 actor in this protocol:

1. All the users and DeFi protocols

- DeFi protocols can use this token as a base token for their protocol for rewards, native token for their protocol etc.

Executive Summary

This audit examined ERC20.sol and supporting scripts to identify security issues that could lead to loss of funds, denial of service, or incorrect token distributions.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	1
Info	0
Total	3

Findings

High

[H-01] Missing Underflow Protection in Internal Burn Enables Supply and Balance Corruption

Description

The ERC20 core implementation contains a critical arithmetic safety flaw in the token burning logic. The `_burn()` function inside `ERC20Internals.sol` performs raw subtraction inside inline assembly without any underflow protection.

If the burn amount exceeds the current user balance or total token supply, the subtraction wraps around due to unchecked EVM arithmetic, producing extremely large values. This permanently corrupts both the global supply accounting and individual balances.

Although `_burn()` is marked as internal, the contract is explicitly designed to be inherited. Any child contract can expose this functionality through a public or external wrapper, unintentionally or maliciously.

This creates a condition where attackers can inflate balances and destroy supply invariants by exploiting arithmetic wraparound.

```
// src/helpers/ERC20Internals.sol
function _burn(address account, uint256 value) internal {
    assembly ("memory-safe") {
        if iszero(account) {
            mstore(0x00, shl(224, 0x96c6fd1e))
            mstore(add(0x00, 4), 0x00)
            revert(0x00, 0x24)
        }

        let supplySlot := _totalSupply.slot
        let balanceSlot := _balances.slot

        // @audit No underflow protection here.
        let supply := sload(supplySlot)
        sstore(supplySlot, sub(supply, value))

        // @audit No underflow protection here.
        let accountBalance := sload(accountBalanceSlot)
        sstore(accountBalanceSlot, sub(accountBalance, value))
    }
}
```

Risk

Likelihood: High

- The contract is designed for inheritance.
- Assembly arithmetic bypasses Solidity 0.8+ safety checks.

- No balance or supply validation exists.

Impact: Critical

- Total supply corruption via underflow.
- User balance inflation through arithmetic wraparound.
- Broken token economics and permanent protocol integrity loss.

Proof of Concept

- The following PoC demonstrates that burning more tokens than available causes underflow and corrupts both balance and total supply.

```
function test_PoC_BurnUnderflow() public {
    address attacker = makeAddr("attacker");

    token.mint(attacker, 1 ether);

    // Burn more than attacker owns
    token.burn(attacker, 2 ether);

    uint256 attackerBalance = token.balanceOf(attacker);
    uint256 supplyAfter = token.totalSupply();

    assertGt(attackerBalance, type(uint256).max / 2);
    assertGt(supplyAfter, type(uint256).max / 2);
}
```

```
forge test --match-test test_PoC_BurnUnderflow -vvv
```

- The test passes, confirming that unchecked subtraction causes state corruption.

Recommended Mitigation

- Add explicit underflow protection before subtraction.

```
function _burn(address account, uint256 value) internal {
+   if (balanceOf(account) < value) revert("Insufficient balance");
+   if (totalSupply < value) revert("Insufficient supply");

    let supply := sload(supplySlot)
    sstore(supplySlot, sub(supply, value))

    let accountBalance := sload(accountBalanceSlot)
    sstore(accountBalanceSlot, sub(accountBalance, value))
}
```

[H-02] Missing Overflow Protection in Internal Mint Allows Arbitrary Token Inflation

Description

The ERC20 core implementation contains a critical arithmetic safety flaw in the token minting logic. The `_mint()` function inside `ERC20Internals.sol` performs raw addition in inline assembly without any overflow protection.

If the mint amount is large enough, adding to the current total supply or user balance can overflow, wrapping around to a small number and effectively resetting balances or supply. This permanently corrupts both the global supply accounting and individual balances.

Although `_mint()` is marked internal, any child contract can expose this functionality through a public or external wrapper, intentionally or accidentally, enabling arbitrary token creation.

```
// src/helpers/ERC20Internals.sol
function _mint(address account, uint256 value) internal {
    assembly ("memory-safe") {
        if iszero(account) {
            mstore(0x00, shl(224, 0xec442f05))
            mstore(add(0x00, 4), 0x00)
            revert(0x00, 0x24)
        }

        let ptr := mload(0x40)
        let balanceSlot := _balances.slot
        let supplySlot := _totalSupply.slot

        // @audit No overflow check here.
        let supply := sload(supplySlot)
        sstore(supplySlot, add(supply, value))

        mstore(ptr, account)
        mstore(add(ptr, 0x20), balanceSlot)
        let accountBalanceSlot := keccak256(ptr, 0x40)
        // @audit No overflow check here.
        let accountBalance := sload(accountBalanceSlot)
        sstore(accountBalanceSlot, add(accountBalance, value))
    }
}
```

Risk

Likelihood: High

- The contract is designed for inheritance.
- Arithmetic is performed unchecked in assembly.

- No validation exists for total supply or account balance overflow.

Impact: Critical

- Arbitrary inflation of user balances.
- Total supply overflow and corruption.
- Broken token economics and permanent protocol integrity loss.

Proof of Concept

- The following PoC demonstrates that minting via a malicious child contract can overflow balances or supply.

```
contract MaliciousToken is ERC20 {
    constructor() ERC20("Evil", "EVL") {}

    function mintAnyone(address to, uint256 amount) public {
        _mint(to, amount);
    }

    function balanceOf(address user) public view override returns
(uint256) {
        return _balanceOf(user);
    }

    function totalSupply() public view override returns (uint256) {
        return totalSupply_();
    }
}

function test_PoC_MintOverflow() public {
    address attacker = makeAddr("attacker");

    MaliciousToken evil = new MaliciousToken();

    // Step 1: Push totalSupply near uint256 max
    uint256 nearMax = type(uint256).max - 10 ether;
    evil.mintAnyone(attacker, nearMax);

    assertEq(evil.totalSupply(), nearMax);

    // Step 2: Mint more to trigger overflow
    evil.mintAnyone(attacker, 100 ether);

    uint256 supplyAfter = evil.totalSupply();
    uint256 balanceAfter = evil.balanceOf(attacker);

    // Supply should have wrapped around to a small number
    assertLt(supplyAfter, 1000 ether);
    assertLt(balanceAfter, 1000 ether);
}
```

```
forge test --match-test test_PoC_MintOverflow -vvv
```

- The test passes, confirming that unchecked addition causes silent overflow and state corruption.

Recommended Mitigation

- Add explicit overflow protection before performing arithmetic inside `_mint()`:

```
function _mint(address account, uint256 value) internal {  
+   require(value > 0, "Invalid amount");  
+   require(totalSupply + value >= totalSupply, "Overflow");  
+   require(balanceOf(account) + value >= balanceOf(account),  
"Overflow");  
  
    let supply := sload(supplySlot)  
    sstore(supplySlot, add(supply, value))  
  
    let accountBalance := sload(accountBalanceSlot)  
    sstore(accountBalanceSlot, add(accountBalance, value))  
}
```

Low

[L-01] Mint and Burn Do Not Emit Required ERC20 Transfer Events

Description

The ERC20 core implementation fails to emit mandatory Transfer events during mint and burn operations.

According to the ERC20 standard, minting must emit a Transfer event from the zero address (0x0) to the recipient, and burning must emit a Transfer event from the holder to the zero address. This contract performs balance and supply updates silently without generating these events.

This breaks ERC20 compliance and causes serious compatibility issues with wallets, indexers, analytics tools, and off-chain infrastructure that depend on events to track token activity.

The issue originates from the internal mint and burn implementations, which update storage directly via inline assembly without emitting logs.


```
// src/helpers/ERC20Internals.sol
function _mint(address account, uint256 value) internal {
    assembly ("memory-safe") {
        // ... validation omitted

        let supply := sload(supplySlot)
        sstore(supplySlot, add(supply, value))

        let accountBalance := sload(accountBalanceSlot)
        sstore(accountBalanceSlot, add(accountBalance, value))

        // Missing: emit Transfer(address(0), account, value)
    }
}

function _burn(address account, uint256 value) internal {
    assembly ("memory-safe") {
        // ... validation omitted

        let supply := sload(supplySlot)
        sstore(supplySlot, sub(supply, value))

        let accountBalance := sload(accountBalanceSlot)
        sstore(accountBalanceSlot, sub(accountBalance, value))

        // Missing: emit Transfer(account, address(0), value)
    }
}
```

Risk

Likelihood: High

- The mint and burn logic is used in core token functionality.
- The contract is designed for inheritance.
- No event emission exists anywhere in these functions.

Impact: Medium

- Off-chain services cannot detect mint and burn activity.
- Wallets and block explorers show incorrect balances.
- Token trackers and DeFi protocols may malfunction.
- Breaks ERC20 standard compliance.

Proof of Concept

- The following test confirms that minting does not emit a Transfer event:

```
import {Vm} from "forge-std/Vm.sol";
```

```
function test_PoC_MintDoesNotEmitTransferEvent() public {
    address user = makeAddr("user");

    vm.recordLogs();
    token.mint(user, 100e18);

    Vm.Log[] memory logs = vm.getRecordedLogs();

    bytes32 transferSig =
keccak256("Transfer(address,address,uint256)");
    bool found;

    for (uint256 i = 0; i < logs.length; i++) {
        if (logs[i].topics.length > 0 && logs[i].topics[0] ==
transferSig) {
            found = true;
        }
    }

    assertEq(found, false);
}
```

```
forge test --match-test test_PoC_MintDoesNotEmitTransferEvent -vvv
```

- This confirms the contract does not emit the required ERC20 event.

Recommended Mitigation

- Add standard ERC20 Transfer event emission inside `_mint()` and `_burn()`.

```
function _mint(address account, uint256 value) internal {
    // balance and supply updates...

+   emit Transfer(address(0), account, value);
}

function _burn(address account, uint256 value) internal {
    // balance and supply updates...

+   emit Transfer(account, address(0), value);
}
```

Description

When the tournament owner sets a winner, the contract later calculates each participant's reward proportionally to their shares using the formula:

```
uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,  
totalWinnerShares);
```

Here, `totalWinnerShares` represents the total shares of users who selected the winning option.

However, if the owner sets a winner that no user has selected, then `totalWinnerShares` becomes 0. Any subsequent call to `withdraw()` or other reward-claiming functions will revert with a division-by-zero error inside `Math.mulDiv`, permanently preventing anyone from claiming or withdrawing funds.

This leads to a complete vault lockup — no participants can withdraw, and the event's assets remain frozen.

```
// briVault.sol  
function withdraw() external {  
    ...  
    // @audit totalWinnerShares can be zero if no one picked the chosen  
winner  
    uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,  
totalWinnerShares);  
    ...  
}
```

Risk

Likelihood: Medium

- The owner can accidentally (or maliciously) set a winner that no one chose.
- The condition is rare but trivial to trigger intentionally.

Impact: High

- Causes a global denial of service: all withdrawals revert.
- Locks all user funds in the vault permanently.
- No user or admin can recover assets unless a separate rescue mechanism exists.

Proof of Concept

Add this test to your `test/briVault.t.sol` suite:

```
function test_NoWinnerPickedDivisionByZero() public {  
    // Two users deposit different amounts  
    vm.startPrank(user1);  
    mockToken.approve(address(briVault), 10 ether);  
    briVault.deposit(10 ether, user1);  
    vm.stopPrank();  
  
    vm.startPrank(user2);
```

```
mockToken.approve(address(briVault), 5 ether);
briVault.deposit(5 ether, user2);
vm.stopPrank();

// Fast-forward to after event end
vm.warp(briVault.eventEndDate() + 1);

// Owner sets a winner that nobody picked
vm.startPrank(briVault.owner());
briVault.setWinner(0); // assume index 0 was never chosen
vm.stopPrank();

// Any withdrawal attempt will now revert due to division by zero
vm.startPrank(user1);
vm.expectRevert(); // Division by zero inside Math.mulDiv
briVault.withdraw();
vm.stopPrank();
}
```

Run with:

```
forge test --match-test test_NoWinnerPickedDivisionByZero -vvv
```

Recommended Mitigation

- Add a guard clause to handle the edge case where no one picked the winning option:

```
function withdraw() external {
    ...
-   uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
+   if (totalWinnerShares == 0) {
+       revert NoWinnerParticipants(); // or refund logic for all
participants
+   }
+   uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
    ...
}
```